RV College of Engineering®

Artificial Intelligence and Machine learning

IS353IA

UNIT 3

Mekhala V Purohit

Assistant Professor

Department of Computer Science and Engineering

RVCE

All material presented in this PowerPoint is credited to the respective authors of standard textbooks, reference presentations shared by senior faculty at **RVCE**, and authentic internet sources.

Note to Students:

This presentation is meant **only for classroom reference and conceptual understanding**.

For exam and detailed preparation, please refer to the **textbook extracts and materials shared in Google Classroom**, as the content here is a **summary version** and not exhaustive.

Go, change the world®

OUTLINE

- **Supervised Learning**
- **Decision Tree Classifier**
- **Model Overfitting**
- **Model Selection**

UNIT-3 Supervised Learning (Part 1)

Basic Concepts, General Framework for Classification

Early History & Progress of Machine learning

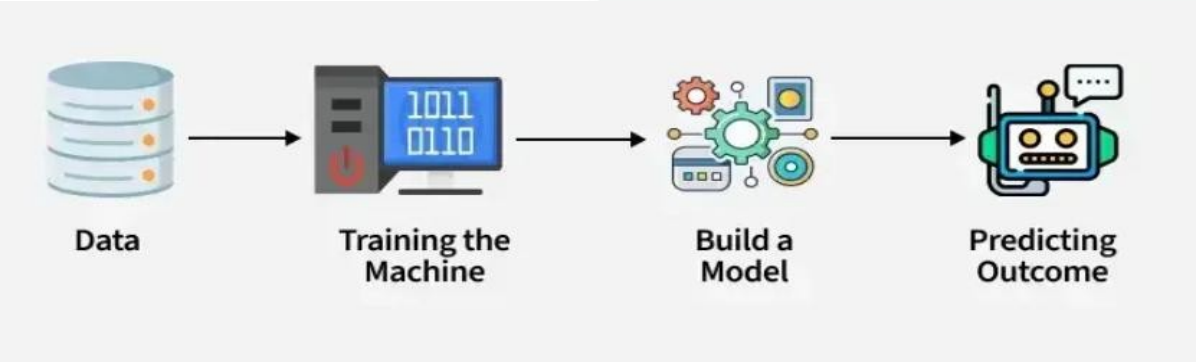
- Since computers were invented, researchers have asked: Can machines learn?
- Machines still cannot learn as well as humans- but progress is significant.
- Effective algorithms now exist for specific learning tasks.

Potential uses:

- Medical diagnosis & treatment recommendations
- Smart homes optimizing energy use
- Personal assistants adapting to user interests

What is Machine Learning?

Machine Learning means teaching computers to learn from examples (data) and make predictions.



Well Posed Learning Problem

A computer program is said to learn from experience E with respect to some class of tasks *T* and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E.

Clearly define three things:

Task (T) — What the program should do.
Experience (E) — How it learns (data, examples, practice).
Performance Measure (P) — How you measure if it's getting better.

Example — Email Spam Filter

Task (T): Classify emails as spam or not spam.
Experience (E): The program learns from examples of emails labeled "spam" or "not spam."
Performance Measure (P): The percentage of emails correctly classified (accuracy).

Try for:

1. Disaster Response AI
2. Heritage Restoration AI

Well Posed Learning Problem

Example: Disaster Response AI

- **Task (T):** Identify and locate disaster-affected areas or survivors using satellite or drone imagery.
- **Experience (E):** Learns from past disaster images labeled with damage levels, affected zones, or human presence.
- **Performance Measure (P):** Accuracy or F1-score in correctly detecting and classifying disaster regions or survivors.

Example — Heritage Restoration AI

- **Task (T):** Restore or reconstruct damaged artworks, manuscripts, or audio using AI-based enhancement techniques.
- **Experience (E):** Trained on large datasets of intact artworks, historical images, or audio to learn restoration patterns.
- **Performance Measure (P):** Similarity between AI-restored output and expert-restored originals (measured via PSNR, SSIM, or expert evaluation).

Well Posed Learning Problem

1. Medical Diagnosis AI
2. Autonomous Driving AI
3. Agriculture Monitoring AI
4. Wildlife Conservation AI
5. Fake News Detection AI
6. Personalized Learning AI
7. Energy Consumption Optimization AI
8. Cybersecurity Threat Detection AI
9. Smart Traffic Management AI
10. Climate Change Prediction AI

Designing a Learning System

Choosing the Training Experience:
Where will the learner get its experience or data? Will it get clear feedback or figure things out on its own?

Choosing the Target Function:A
What exactly do we want the learner to predict or decide? This is the goal or "rule" we want it to discover.

Choosing a Representation for the Target Function:
How will the learner store or express what it has learned? (e.g., decision trees, rules, neural networks, etc.)

Choosing a Function Approximation Algorithm:
What learning method or algorithm will we use to find the target function from the training examples?

Choosing a Training Experience

Direct vs Indirect Feedback

Direct Feedback:

The program gets clear, exact guidance.

Example: Someone tells the program the **best move** for each board position.

Easy to learn — the program knows exactly what's right or wrong.

Indirect Feedback:

The program only sees whether it **won or lost** the whole game.

*It has to figure out **which moves were good or bad** by itself.*

Harder — because sometimes a good move might still lead to losing if later moves are bad.

Who Controls the Training Examples?.

Teacher-controlled: A human teacher shows important situations and the best moves.

Learner-controlled (Self-play)

Does the Training Look Like Real-World Testing?

Choosing the right training experience means: Give clear feedback if possible. Make sure the learner sees examples similar to what it will see in the real world. Decide who controls what it learns from — a teacher, the learner itself, or both.

Choosing a Target Function

When you design a learning system, you must decide: What exactly do you want the program to learn to do? You want to build a computer program that predicts how much a house will sell for.

Task (T): Predict house price

Experience (E): Past data about houses — size, location, number of bedrooms, and the actual price they sold for.

Performance (P): How close the predicted price is to the real selling price.

Direct Way: Predict the Exact Price

The obvious goal is:

Input: Details about the house **Output:** Predicted price in dollars

Target function is:

Price: H→R where H is the set of all possible houses (with features like size, location, etc.), and R means it outputs a real number (the price).

Choosing a Target Function

But what if direct prediction is too complex?

Suppose the housing market is huge and prices depend on complex factors- neighborhood trends, school ratings, upcoming development projects — some of which are hard to model directly.

In that case, you might simplify the target function: Instead of predicting the exact price

Predict a score: how desirable is the house?

Desirability→[0,100]

Choosing a representation for the Target Function

Spam Email Classifier

Target function: Is this email spam or not?

Set of Rules:

"IF email contains 'win lottery' AND has an unknown sender THEN mark as spam."

Decision Tree:

A tree where you ask questions:
Does the subject contain 'Congratulations'?
Does the sender have a trusted domain?
What is the probability of spam?

Linear Model:

A mathematical formula like:

Spam Score=2.3×(Has 'free')+1.1×(Contains link)−0.5×(Sender known)

Choosing a Function Approximation Algorithm

Decide **what you want to learn** (the target function)
Decide **how you'll represent it** (rules, decision tree, neural network, etc.)
…the next big question is:
HOW will you actually *learn* it from the data?
This is where the **Function Approximation Algorithm** comes in.
A **Function Approximation Algorithm** is the *learning method* that adjusts your chosen representation to fit the training data.
Function approximation" means adjusting model parameters (weights, coefficients, etc.) so that the function best fits the training data.
it finds the parameters that minimize the error between predicted and actual values.

If your representation is a decision tree → you need a tree-learning algorithm.
If your representation is a neural network → you need an algorithm that adjusts the network's weights.
If it's a line or curve → you need an algorithm that finds the best line or curve

Adjusting Weights

$$E = \sum (true - predicted)^2$$

For each training example, you know:

- The true value (from training data)
- The predicted value (from your model)

How does ML Work?

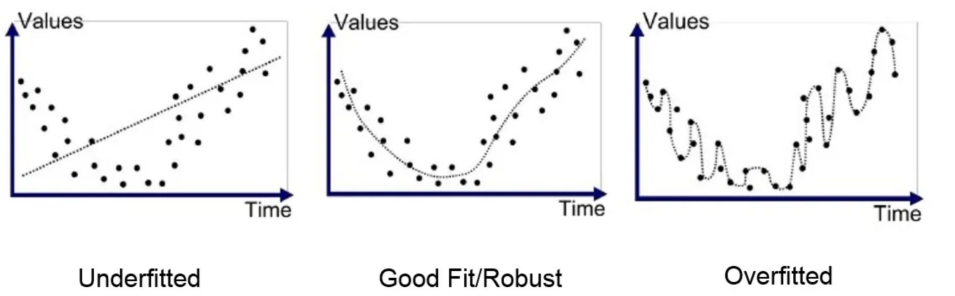
1. Collect Data: Gather examples (numbers, text, images, etc.)
2. Train Model: Show examples to the computer to learn patterns.
3. Test Model: Check learning using new examples.
4. Use Model: Apply it to real-life predictions.

Example: Like teaching a child: show many apples and bananas, test with a new fruit.
Input Data = The examples you give (like pictures of fruits, sounds, or text).
Labels = The correct answers for each example (like “apple” or “banana”).
Learning = The machine looks at the data again and again to find patterns.
Prediction = After learning, it tries to guess new examples by applying what it learned.

- **Underfitting:** The model is *too simple* to capture the underlying pattern in the data.
- **Overfitting:** The model is *too complex* and captures *noise* or random fluctuations instead of the actual pattern.

How does ML Work?

Concept	Analogy
Underfitting	The system only looks at overall head shape — fails to recognize most people.
Overfitting	The system memorizes specific lighting and background — can’t recognize the same person in a new photo.
Good Fit	The system learns key features like eyes, nose, and mouth — recognizes people reliably in new photos.



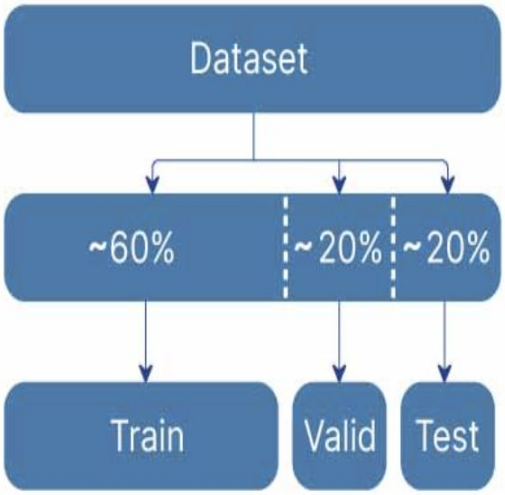
How does ML Work?

We divide data into two parts:

- Training Set (80%): Used to teach the model.
- T est Set (20%): Used to check learning.
- .

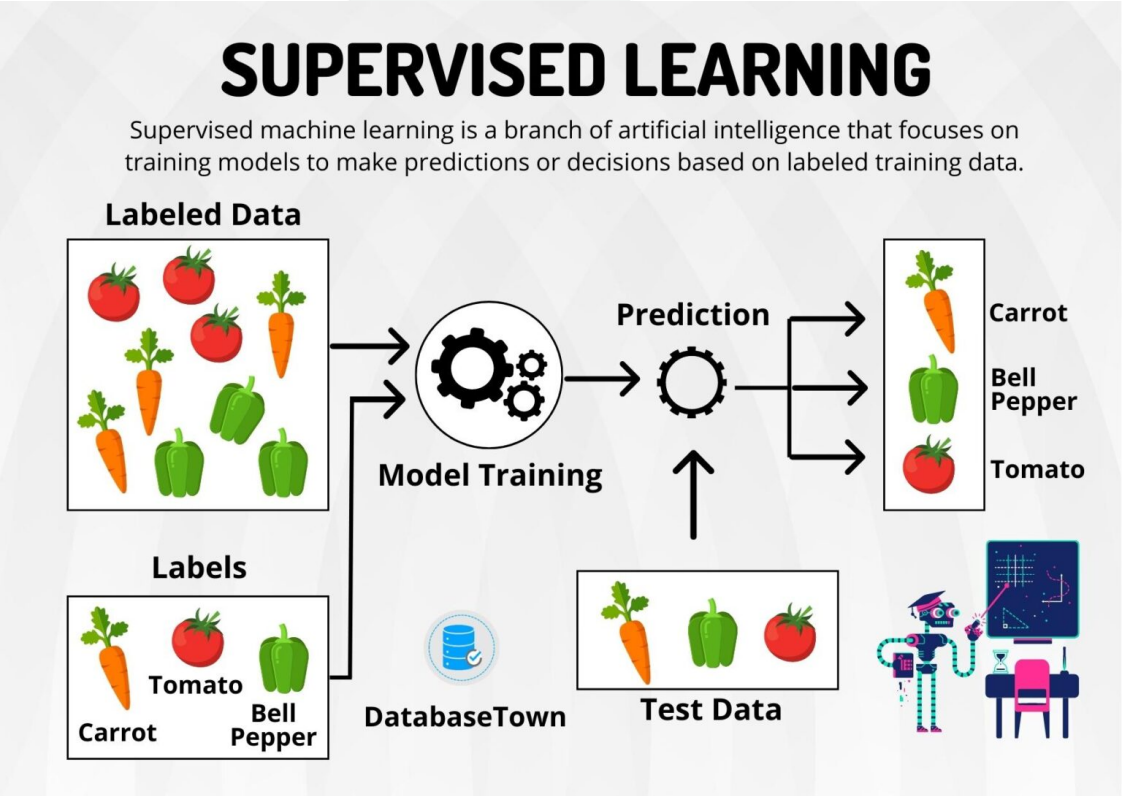
If we train and test on same data, model may memorize answers. We want the model to understand patterns, not memorize.

Example: Reading answers before exam vs truly understanding the topic.



SUPERVISED LEARNING

Introduction

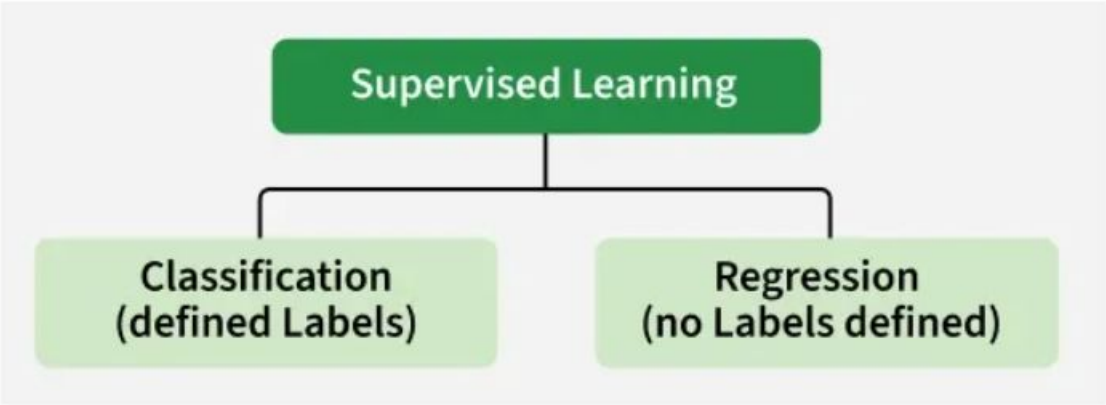


Basic Concepts

Types of Supervised Learning in Machine Learning

Now, Supervised learning can be applied to two main types of problems:

- **Classification:** Where the output is a categorical variable (e.g., spam vs. non-spam emails, yes vs. no).
- **Regression:** Where the output is a continuous variable (e.g., predicting house prices, stock prices, weather forecasting).



Advantages

Here are some advantages of supervised learning listed below:

- **Simplicity & clarity:** Easy to understand and implement since it learns from labeled examples.
- **High accuracy**: When sufficient labeled data is available, models achieve strong predictive performance.
- **Versatility**: Works for both classification like spam detection, disease prediction and regression like price forecasting.
- **Generalization**: With enough diverse data and proper training, models can generalize well to unseen inputs.
- **Wide application**: Used in speech recognition, medical diagnosis, sentiment analysis, fraud detection and more.

Disadvantages

- **Requires labeled data**: Large amounts of labeled datasets are expensive and time-consuming to prepare.
- **Bias from data**: If training data is biased or unbalanced, the model may learn and amplify those biases.
- **Overfitting risk**: Model may memorize training data instead of learning general patterns, especially with small datasets.
- **Limited adaptability**: Performance drops significantly when applied to data distributions very different from training data.
- **Not scalable for some problems**: In tasks with millions of possible labels like natural language, supervised labeling becomes impractical.

Practical Examples of Supervised learning

Few practical examples of supervised machine learning across various industries:

- **Fraud Detection in Banking**: Utilizes supervised learning algorithms on historical transaction data, training models with labeled datasets of legitimate and fraudulent transactions to accurately predict fraud patterns.
- **Parkinson Disease Prediction:** Parkinson's disease is a progressive disorder that affects the nervous system and the parts of the body controlled by the nerves.
- **Customer Churn Prediction:** Uses supervised learning techniques to analyze historical customer data, identifying features associated with churn rates to predict customer retention effectively.
- **Cancer cell classification:** Implements supervised learning for cancer cells based on their features and identifying them if they are 'malignant' or 'benign.
- **Stock Price Prediction**: Applies supervised learning to predict a signal that indicates whether buying a particular stock will be helpful or not.

Working of Supervised Machine Learning

The working of supervised machine learning follows these key steps:

1. Collect Labeled Data

- Gather a dataset where each input has a known correct output (label).
- **Example**: Images of handwritten digits with their actual numbers as labels.

2. Split the Dataset

- Divide the data into training data (about 80%) and testing data (about 20%).
- The model will learn from the training data and be evaluated on the testing data.

3. Train the Model

- Feed the training data (inputs and their labels) to a suitable supervised learning algorithm (like Decision Trees, SVM or Linear Regression).
- The model tries to find patterns that map inputs to correct outputs.

4. Validate and Test the Model

- Evaluate the model using testing data it has never seen before.
- The model predicts outputs and these predictions are compared with the actual labels to calculate accuracy or error.

5. Deploy and Predict on New Data

- Once the model performs well, it can be used to predict outputs for completely new, unseen data.

Basic Concepts

Given a collection of records (training set)

Each record is by characterized by a tuple ($\boldsymbol{x}$,y), where $\boldsymbol{x}$ is the attribute set and y is the class label

- $\boldsymbol{x}$: attribute, predictor, independent variable, input
- y: class, response, dependent variable, output

Task:

Learn a model that maps each attribute set $\boldsymbol{x}$ into one of the predefined class labels y

Task	Attribute set, $\boldsymbol{x}$	Class label, y
Categorizing email messages	Features extracted from email message header and content	spam or non-spam
Identifying tumor cells	Features extracted from x-rays or MRI scans	malignant or benign cells
Cataloging galaxies	Features extracted from telescope images	Elliptical, spiral, or irregular-shaped galaxies

Supervised Machine Learning Algorithms

Supervised learning can be further divided into several different types, each with its own unique characteristics and applications. Here are some of the most common types of supervised learning algorithms:

- **Linear Regression:** Linear regression is a type of supervised learning regression algorithm that is used to predict a continuous output value. It is one of the simplest and most widely used algorithms in supervised learning.
- **Logistic Regression**: Logistic regression is a type of supervised learning classification algorithm that is used to predict a binary output variable.
- **Decision Trees** : Decision tree is a tree-like structure that is used to model decisions and their possible consequences. Each internal node in the tree represents a decision, while each leaf node represents a possible outcome.
- **Random Forests**: Random forests again are made up of multiple decision trees that work together to make predictions. Each tree in the forest is trained on a different subset of the input features and data. The final prediction is made by aggregating the predictions of all the trees in the forest.

Basic Concepts

Supervised Machine Learning Algorithms

- **Support Vector Machine(SVM):** The SVM algorithm creates a hyperplane to segregate n-dimensional space into classes and identify the correct category of new data points. The extreme cases that help create the hyperplane are called support vectors, hence the name Support Vector Machine.
- **K-Nearest Neighbors:** KNN works by finding k training examples closest to a given input and then predicts the class or value based on the majority class or average value of these neighbors. The performance of KNN can be influenced by the choice of k and the distance metric used to measure proximity.
- **Gradient Boosting:** Gradient Boosting combines weak learners, like decision trees, to create a strong model. It iteratively builds new models that correct errors made by previous ones.
- **Naive Bayes Algorithm:** The Naive Bayes algorithm is a supervised machine learning algorithm based on applying Bayes' Theorem with the “naive” assumption that features are independent of each other given the class label.

Classification Techniques

Base Classifiers
Decision Tree based Methods

Unit 4
Rule-based Methods
Nearest-neighbor
Naïve Bayes and Bayesian Belief Networks
Support Vector Machines
Neural Networks, Deep Neural Nets

Ensemble Classifiers(Unit 4)
Boosting, Bagging, Random Forests

Decision Tree Introduction

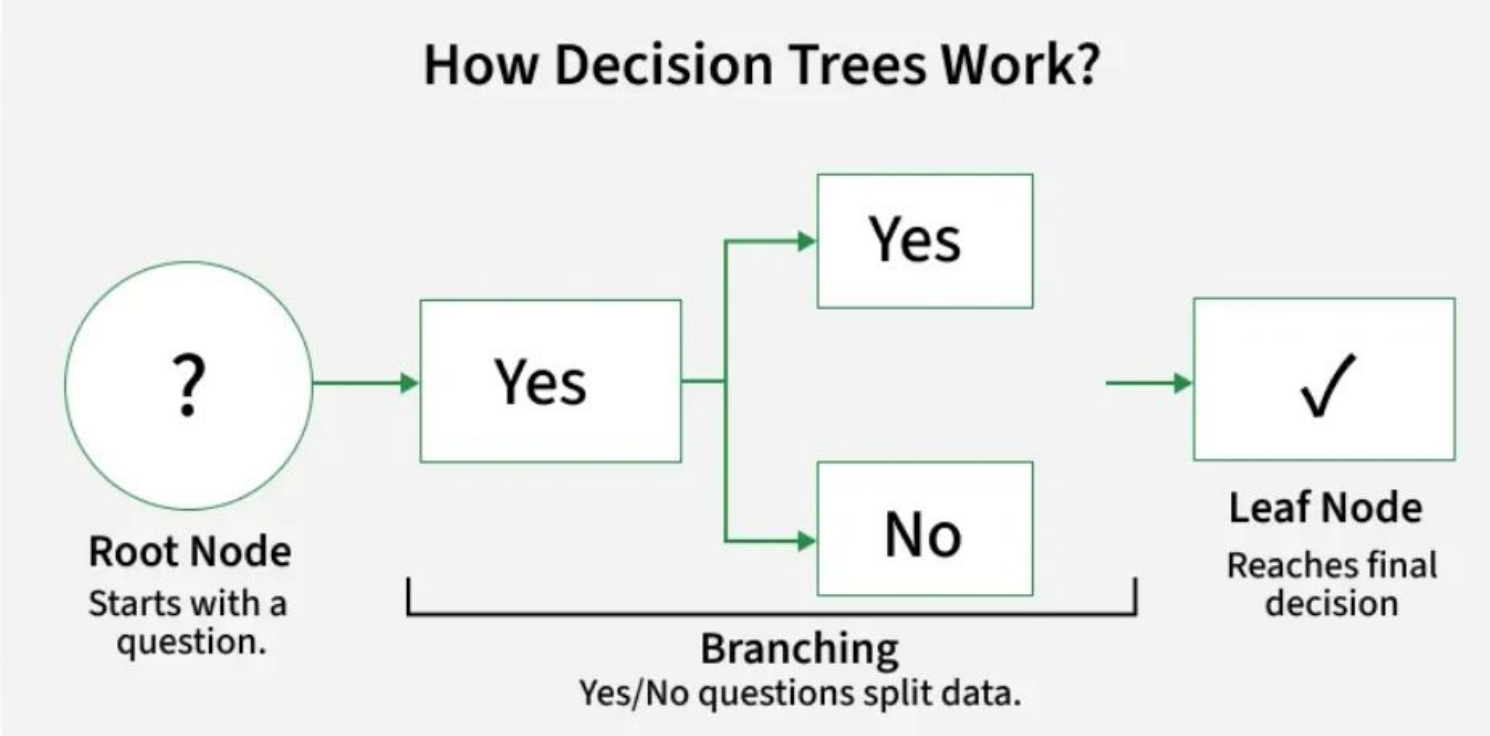
Decision tree learning is a method for approximating **discrete-valued target functions**, in which the learned function is represented as a **tree structure**. Each internal node of the tree corresponds to a test on an attribute, each branch corresponds to an outcome of that test, and each leaf node represents a class label (the value of the target function).

The process of learning a decision tree is one of the most widely used and practical methods of **inductive inference** — that is, learning general concepts from specific examples. Decision trees are intuitive and interpretable, as they can be easily converted into a set of **if–then rules**, improving their readability for humans.

Structure:

- **Root Node:** Represents the full dataset.
- **Internal Nodes:** Each node will have some kind of associated atricute. Attribute tests(Points where decisions are made based on data features.)
- **Branches:** Corresponds to one of the possible values for this attribute(Lines connecting nodes showing the flow from one decision to another.)
- **Leaf Nodes:** Final class labels or decisions.

Decision Tree Introduction



Example: Play Tennis Decision Tree

1. Purpose:

•The tree is used to decide **whether to play tennis** on a Saturday morning.

2. Attributes Used:

•**Outlook** (Sunny, Overcast, Rain)
•**Temperature** (Hot, Mild, Cool)
•**Humidity** (High, Normal)
•**Wind** (Weak, Strong)

3. Working Example:

•Input instance:
(Outlook = Sunny, Temperature = Hot, Humidity = High, Wind = Strong)
•The instance follows the **leftmost path** in the tree.
•**Predicted result:** PlayTennis = **No**

APPROPRIATE PROBLEMS FOR DECISION TREE LEARNING

Although a variety of decision tree learning methods have been developed with somewhat differing capabilities and requirements, decision tree learning is generally best suited to problems with the following characteristics:

1. Attribute–Value Representation:

- Instances are described using **attributes and their values**.
  (Example: Temperature = Hot)
- Works best when each attribute has a **few distinct values**.
- Can also handle **numeric attributes** with extensions.

2. Discrete Target Function:

- Output is usually **discrete** (e.g., *Yes / No*).
- Can handle **multi-class outputs** (more than two values).
- Extensions allow **real-valued outputs**, but less common.

3. Handles Disjunctive Descriptions:

- Decision trees can naturally represent **"OR" conditions** (disjunctions).
  (Example: Play = Yes if Overcast OR Humidity = Normal)

4. Tolerant to Errors:

- Works well even if training data has some **wrong labels** or **attribute errors**.
- Robust to noisy data.

5. Handles Missing Values:

- Can deal with **unknown or missing attributes** in training data. *(Example: Humidity value missing for some samples)*

Example of a Decision Tree

The **goal** of this Decision Tree is to **predict whether a borrower will default on a loan or not** that's the **"Defaulted Borrower"** column (Yes / No) on the left.
So the tree is learning **rules** that map input features like:

- Home Ownership
- Marital Status
- Annual Income

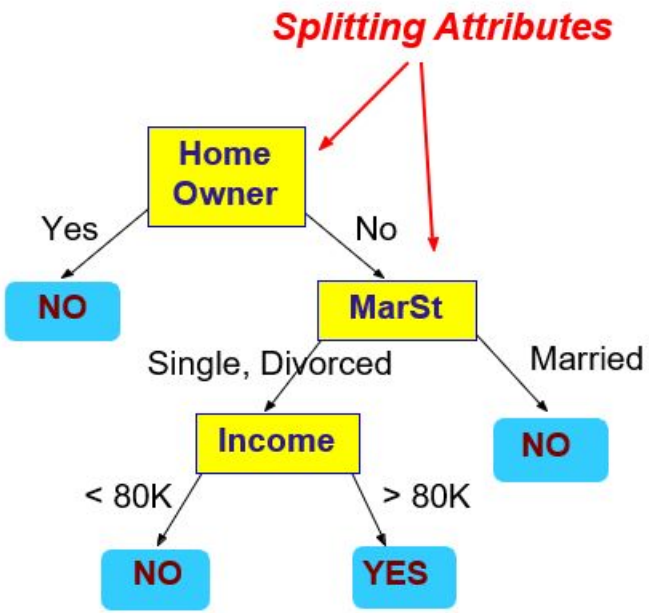
to the **decision (output) → "Will the borrower default?"**

- **Defaulted = Yes** → the borrower **did not pay back** the loan.
- **Defaulted = No** → the borrower **successfully repaid** the loan.

Example of a Decision Tree

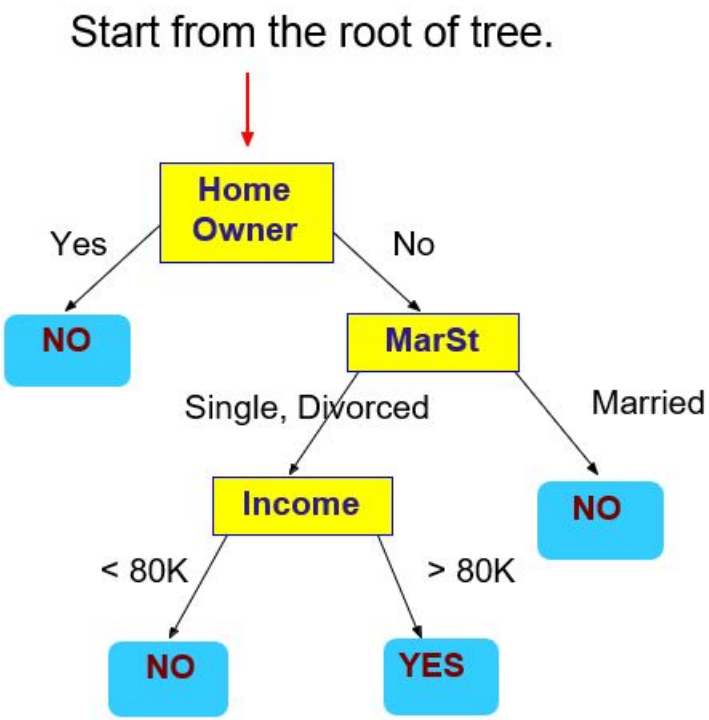
ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
	categorical	categorical	continuous	class
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Training Data



Model: Decision Tree

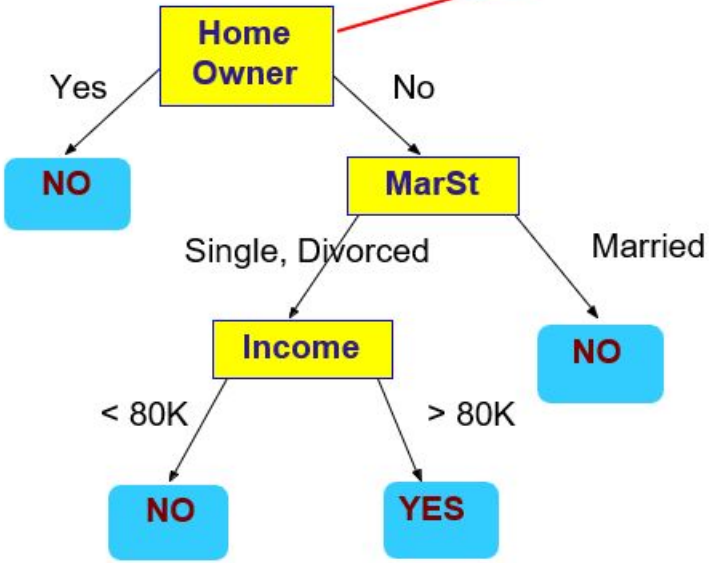
Example of a Decision Tree



Test Data

Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?

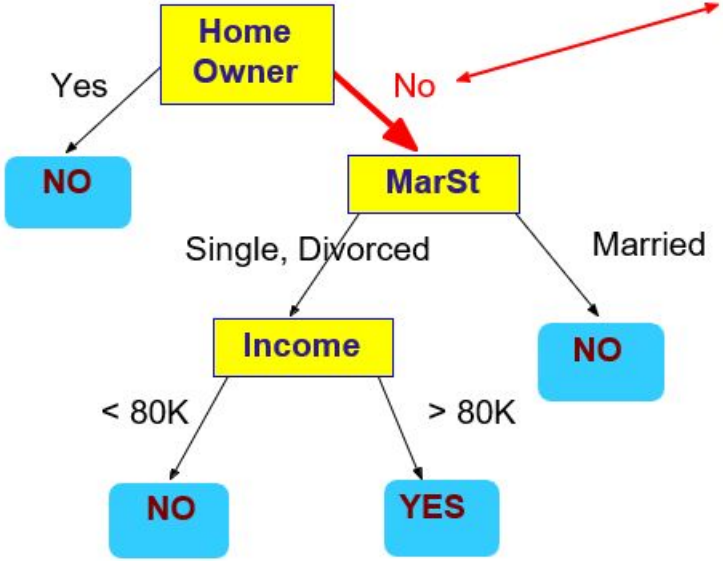
Example of a Decision Tree



Test Data

Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?

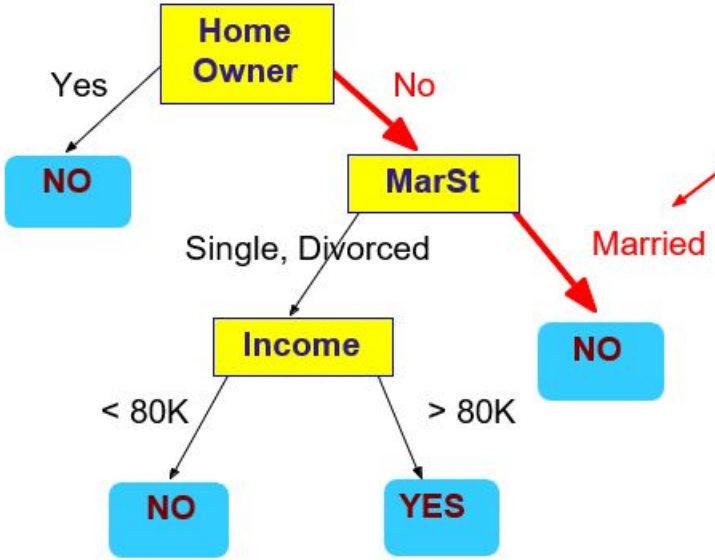
Example of a Decision Tree



Test Data

Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?

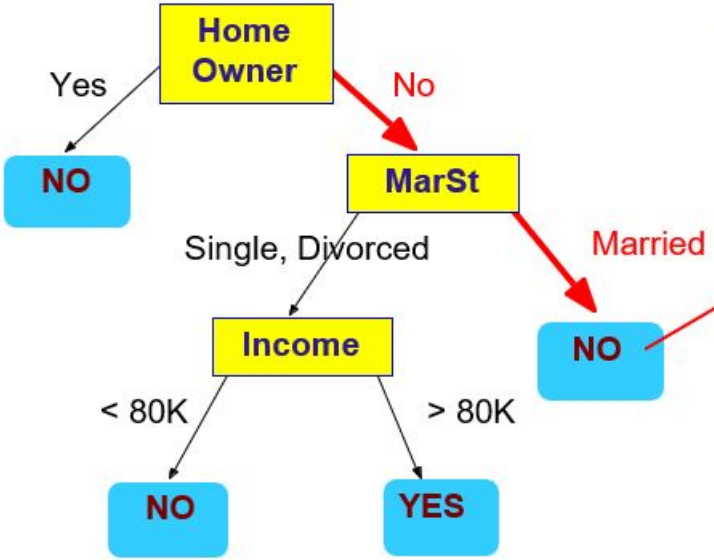
Example of a Decision Tree



Test Data

Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?

Example of a Decision Tree



Test Data

Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?

Assign Defaulted to "No"

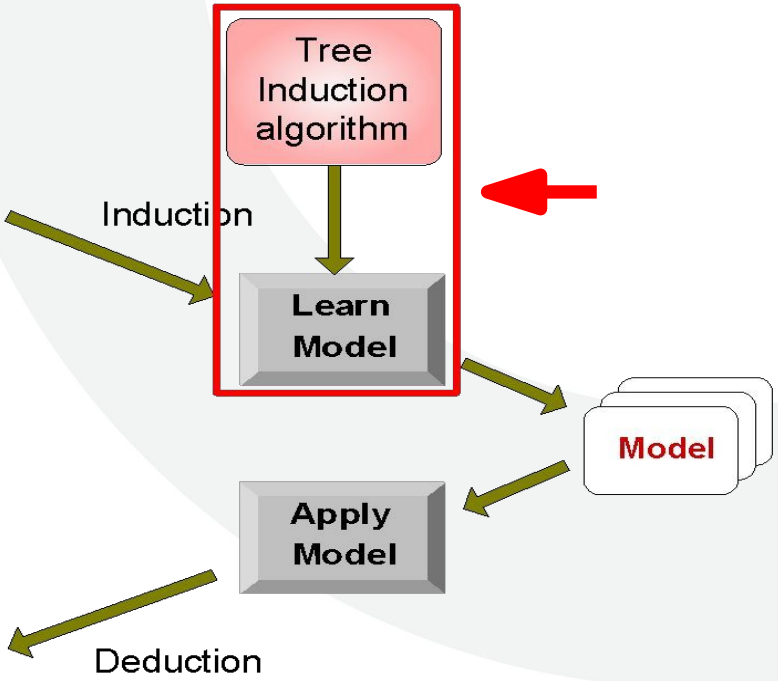
Decision Tree Classification Structure

Tid	Attrib1	Attrib2	Attrib3	Class
1	Yes	Large	125K	No
2	No	Medium	100K	No
3	No	Small	70K	No
4	Yes	Medium	120K	No
5	No	Large	95K	Yes
6	No	Medium	60K	No
7	Yes	Large	220K	No
8	No	Small	85K	Yes
9	No	Medium	75K	No
10	No	Small	90K	Yes

Training Set

Tid	Attrib1	Attrib2	Attrib3	Class
11	No	Small	55K	?
12	Yes	Medium	80K	?
13	Yes	Large	110K	?
14	No	Small	95K	?
15	No	Large	67K	?

Test Set



Hunt's Algorithm — Foundation

Hunt's Algorithm is **not a specific decision tree algorithm like ID3, C4.5, or CART**.
It is the **foundational procedure or framework** that *all those algorithms are built upon*.

What Hunt's Algorithm Defines

It describes **how to recursively build a tree**:

1. **Start** with all training examples at the root.
2. **If all examples belong to one class** → make a leaf node.
3. **Else:**
    1. Choose the **best attribute** to split on (using a measure like entropy, Gini, etc.).
    2. **Split** the dataset into subsets.
    3. **Repeat** the process for each subset (recursion).

So the "structure" of modern decision tree algorithms follows Hunt's recursive logic.

Few common Decision Tree Algorithms

Algorithm	Description	Key Feature
ID3 (Iterative Dichotomiser 3)	Introduced by Ross Quinlan (1986). Uses **Information Gain (Entropy)** for splitting.	Works only with **categorical features**.
C4.5	Successor to ID3 (by Quinlan). Uses **Gain Ratio** instead of plain Information Gain.	Handles **continuous data**, **missing values**, and **pruning**.
C5.0	Commercial version of C4.5 (by Quinlan).	Faster and more memory-efficient; supports **boosting**.
CART (Classification and Regression Tree)	By Breiman et al. Uses **Gini Index** (for classification) or **MSE** (for regression).	Can handle **both categorical and numerical** data.

Entropy and Information Gain

Entropy is a measure of impurity or randomness in a dataset.

It is defined as:

$$Entropy(S) \equiv -p_{\oplus} \log_2 p_{\oplus} - p_{\ominus} \log_2 p_{\ominus}$$

In Simple Words:

Entropy tells **how uncertain or mixed the data is**

- If all examples belong to one class → entropy = 0 (pure).
- If classes are evenly split → entropy = 1 (impure).

Example:

If 9 samples are "Yes" and 5 are "No":

$$Entropy = -[(9/14)\log_2(9/14) + (5/14)\log_2(5/14)] = 0.94$$

Entropy and Information Gain

Information Gain measures the **expected reduction in entropy** caused by partitioning the dataset based on an attribute A:

$$InformationGain(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} \times Entropy(S_v)$$

where S_v is the subset of S for which attribute A takes value v.

In Simple Words:

Information Gain shows **how much uncertainty is reduced** after splitting the data using a feature. The feature with the **highest Information Gain** is chosen for the split.

Play Tennis	Count
Yes	2
No	3

Entropy and Information Gain

Entropy after splitting by "Weather"

Weather	Play Tennis (Yes/No)	Entropy
Sunny	0 Yes, 2 No	0.0
Overcast	1 Yes, 0 No	0.0
Rain	1 Yes, 1 No	1.0

$$Entropy(Parent) = -\left(\frac{2}{5}\log_2\frac{2}{5} + \frac{3}{5}\log_2\frac{3}{5}\right) = 0.971$$

- Sunny: $2/5$
- Overcast: $1/5$
- Rain: $2/5$

$$Entropy(Children) = \frac{2}{5}(0) + \frac{1}{5}(0) + \frac{2}{5}(1) = 0.4$$

$$IG(Weather) = 0.971 - 0.4 = 0.571$$

ID3 Algorithm

ID3(*Examples*, *Target_attribute*, *Attributes*)

Examples are the training examples. Target_attribute is the attribute whose value is to be predicted by the tree. Attributes is a list of other attributes that may be tested by the learned decision tree. Returns a decision tree that correctly classifies the given Examples.

- Create a *Root* node for the tree
- If all *Examples* are positive, Return the single-node tree *Root*, with label = +
- If all *Examples* are negative, Return the single-node tree *Root*, with label = −
- If *Attributes* is empty, Return the single-node tree *Root*, with label = most common value of *Target_attribute* in *Examples*
- Otherwise Begin
  - $A \leftarrow$ the attribute from *Attributes* that best* classifies *Examples*
  - The decision attribute for *Root* $\leftarrow A$
  - For each possible value, v_i, of A,
    - Add a new tree branch below *Root*, corresponding to the test $A = v_i$
    - Let $Examples_{v_i}$ be the subset of *Examples* that have value v_i for A
    - If $Examples_{v_i}$ is empty
      - Then below this new branch add a leaf node with label = most common value of *Target_attribute* in *Examples*
      - Else below this new branch add the subtree
        ID3($Examples_{v_i}$, *Target_attribute*, *Attributes* − {A}))
- End
- Return *Root*

Building Tree using ID3 Algorithm

Day	*Outlook*	*Temperature*	*Humidity*	*Wind*	*PlayTennis*
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Building Tree using ID3 Algorithm

Attribute: Outlook

Values (Outlook) = Sunny, Overcast, Rain

$$S = [9^+, 5^-]$$

$$Entropy(S) = -\frac{9}{14}\log_2\frac{9}{14} - \frac{5}{14}\log_2\frac{5}{14} = 0.9403$$

Subsets:

$$S_{Sunny} = [2^+, 3^-], \qquad S_{Overcast} = [4^+, 0^-], \qquad S_{Rain} = [3^+, 2^-]$$

$$Entropy(S_{Sunny}) = -\frac{2}{5}\log_2\frac{2}{5} - \frac{3}{5}\log_2\frac{3}{5} = 0.9710$$

$$Entropy(S_{Overcast}) = -\frac{4}{4}\log_2\frac{4}{4} - \frac{0}{4}\log_2\frac{0}{4} = 0.0$$

$$Entropy(S_{Rain}) = -\frac{3}{5}\log_2\frac{3}{5} - \frac{2}{5}\log_2\frac{2}{5} = 0.9710$$

Information Gain (Outlook):

$$Gain(S, \text{Outlook}) = Entropy(S) - \sum_{v \in \{\text{Sunny,Overcast,Rain}\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$= 0.9403 - \frac{5}{14}(0.9710) - \frac{4}{14}(0.0) - \frac{5}{14}(0.9710)$$

$$= 0.9403 - \frac{10}{14}(0.9710)$$

$$\boxed{Gain(S, \text{Outlook}) = 0.2467}$$

Building Tree using ID3 Algorithm

- **Attribute: Temp**

Values (Temp) = Hot, Mild, Cool

$$S = [9^+, 5^-]$$

$$Entropy(S) = -\frac{9}{14}\log_2\frac{9}{14} - \frac{5}{14}\log_2\frac{5}{14} = 0.94$$

Subsets

$$S_{Hot} = [2^+, 2^-], \quad S_{Mild} = [4^+, 2^-], \quad S_{Cool} = [3^+, 1^-]$$

$$Entropy(S_{Hot}) = -\frac{2}{4}\log_2\frac{2}{4} - \frac{2}{4}\log_2\frac{2}{4} = 1.0$$

$$Entropy(S_{Mild}) = -\frac{4}{6}\log_2\frac{4}{6} - \frac{2}{6}\log_2\frac{2}{6} = 0.9183$$

$$Entropy(S_{Cool}) = -\frac{3}{4}\log_2\frac{3}{4} - \frac{1}{4}\log_2\frac{1}{4} = 0.8113$$

Information Gain

$$Gain(S, Temp) = Entropy(S) - \sum_{v \in \{Hot, Mild, Cool\}} \frac{|S_v|}{|S|} \times Entropy(S_v)$$

$$Gain(S, Temp) = 0.94 - \frac{4}{14}(1.0) - \frac{6}{14}(0.9183) - \frac{4}{14}(0.8113)$$

$$\boxed{Gain(S, Temp) = 0.029}$$

Building Tree using ID3 Algorithm

Attribute: Humidity
Values (Humidity) = *High, Normal*

For Humidity = High

$$S_{High} = [3^+, 4^-]$$

$$Entropy(S_{High}) = -\frac{3}{7}\log_2\frac{3}{7} - \frac{4}{7}\log_2\frac{4}{7} = 0.9852$$

For Humidity = Normal

$$S_{Normal} = [6^+, 1^-]$$

$$Entropy(S_{Normal}) = -\frac{6}{7}\log_2\frac{6}{7} - \frac{1}{7}\log_2\frac{1}{7} = 0.5916$$

Information Gain

$$Gain(S, Humidity) = Entropy(S) - \sum_{v \in \{High, Normal\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S, Humidity) = Entropy(S) - \frac{7}{14} Entropy(S_{High}) - \frac{7}{14} Entropy(S_{Normal})$$

$$Gain(S, Humidity) = 0.94 - \frac{7}{14}(0.9852) - \frac{7}{14}(0.5916) = 0.1516$$

Building Tree using ID3 Algorithm

Attribute: Wind

Values (Wind) = *Strong, Weak*

Given (dataset)

$$S = [9^+,\ 5^-] \quad \Rightarrow \quad Entropy(S) = -\frac{9}{14}\log_2\frac{9}{14} - \frac{5}{14}\log_2\frac{5}{14} = 0.94$$

Split by Wind

- Wind = Strong

$$S_{Strong} = [3^+,\ 3^-] \quad (|S_{Strong}| = 6)$$

$$Entropy(S_{Strong}) = -\frac{3}{6}\log_2\frac{3}{6} - \frac{3}{6}\log_2\frac{3}{6} = -\tfrac{1}{2}\log_2\tfrac{1}{2} - \tfrac{1}{2}\log_2\tfrac{1}{2} = 1.0$$

- Wind = Weak

$$S_{Weak} = [6^+,\ 2^-] \quad (|S_{Weak}| = 8)$$

$$Entropy(S_{Weak}) = -\frac{6}{8}\log_2\frac{6}{8} - \frac{2}{8}\log_2\frac{2}{8} = -\tfrac{3}{4}\log_2\tfrac{3}{4} - \tfrac{1}{4}\log_2\tfrac{1}{4} = 0.8113$$

Information Gain (Wind)

$$Gain(S, \text{Wind}) = Entropy(S) - \frac{|S_{Strong}|}{|S|}Entropy(S_{Strong}) - \frac{|S_{Weak}|}{|S|}Entropy(S_{Weak})$$

Plug numbers:

$$Gain(S, \text{Wind}) = 0.94 - \frac{6}{14}\cdot 1.0 - \frac{8}{14}\cdot 0.8113$$

$$\frac{6}{14} = 0.428571, \quad \frac{8}{14} = 0.571429$$

$$Gain(S, \text{Wind}) = 0.94 - 0.428571 - 0.571429 \times 0.8113 = 0.94 - 0.428571 - 0.463728 \approx 0.0478$$

Building Tree using ID3 Algorithm

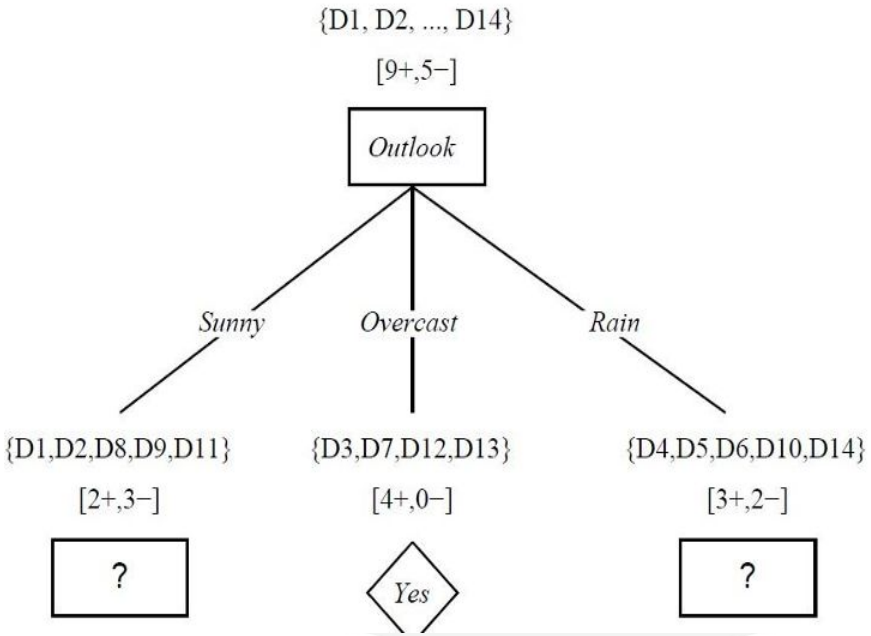
Information Gain Calculations

$$Gain(S, Outlook) = 0.2464$$

$$Gain(S, Temp) = 0.0289$$

$$Gain(S, Humidity) = 0.1516$$

$$Gain(S, Wind) = 0.0478$$



Building Tree using ID3 Algorithm

We already chose:

$$\textbf{Root: Outlook} \quad (\text{Gain} = 0.2467)$$

Branches:

- Overcast → all positive → **leaf: Play = Yes**.
- Sunny → subset $S_{Sunny} = [2^+, 3^-]$ (5 examples) → not pure → choose next attribute.
- Rain → subset $S_{Rain} = [3^+, 2^-]$ (5 examples) → not pure → choose next attribute.

Day	Temp	Humidity	Wind	Play Tennis
D1	Hot	High	Weak	**No**
D2	Hot	High	Strong	**No**
D8	Mild	High	Weak	**No**
D9	Cool	Normal	Weak	**Yes**
D11	Mild	Normal	Strong	**Yes**

Building Tree using ID3 Algorithm

Next attribute for the **Sunny** branch

Total Records in S_{Sunny} ($|S|$): **5**

Positive (Yes) Instances: **2**

Negative (No) Instances: **3**

$S_{Sunny} \rightarrow$ **[2+, 3-]**

Entropy measures the impurity of the current set of examples.

$$\text{Entropy}(S) = -p_{\text{yes}} \log_2(p_{\text{yes}}) - p_{\text{no}} \log_2(p_{\text{no}})$$

- **Calculation:**

$$\text{Entropy}(S_{Sunny}) = -\frac{2}{5} \log_2\left(\frac{2}{5}\right) - \frac{3}{5} \log_2\left(\frac{3}{5}\right)$$

$$\text{Entropy}(S_{Sunny}) = -0.4(-1.32) - 0.6(-0.74)$$

$$\text{Entropy}(S_{Sunny}) = 0.528 + 0.444 \approx \mathbf{0.97}$$

Building Tree using ID3 Algorithm

Attribute: Temperature on S_{Sunny}

Subset $S_{Sunny} = [2^+, 3^-]$, $|S_{Sunny}| = 5$.

A. Entropy of each branch

Temperature = Hot (S_{Hot}): Days D1, D2 → [0⁺, 2⁻] → $|S_{Hot}| = 2$

$$Entropy(S_{Hot}) = -\frac{0}{2}\log_2\frac{0}{2} - \frac{2}{2}\log_2\frac{2}{2} = 0 - 1(0) = \mathbf{0.0}$$

Temperature = Mild (S_{Mild}): Days D8, D9 → [1⁺, 1⁻] → $|S_{Mild}| = 2$

$$Entropy(S_{Mild}) = -\frac{1}{2}\log_2\frac{1}{2} - \frac{1}{2}\log_2\frac{1}{2} = 1.0$$

Temperature = Cool (S_{Cool}): Day D11 → [1⁺, 0⁻] → $|S_{Cool}| = 1$

$$Entropy(S_{Cool}) = -\frac{1}{1}\log_2\frac{1}{1} - \frac{0}{1}\log_2\frac{0}{1} = \mathbf{0.0}$$

B. Calculate $Gain(S_{Sunny}, Temperature)$

$$Gain(S_{Sunny}, Temperature) = Entropy(S_{Sunny}) - \left[\frac{|S_{Hot}|}{|S|}Entropy(S_{Hot}) + \frac{|S_{Mild}|}{|S|}Entropy(S_{Mild}) + \frac{|S_{Cool}|}{|S|}Entropy(S_{Cool})\right]$$

Substitute values:

$$= 0.97 - \left[\frac{2}{5}\cdot 0 + \frac{2}{5}\cdot 1.0 + \frac{1}{5}\cdot 0\right]$$

$$= 0.97 - 0.40 = \boxed{\mathbf{0.57}}$$

Building Tree using ID3 Algorithm

Attribute: Wind on S(Sunny)

Subset $S_{Sunny} = [2^+, 3^-]$, $|S_{Sunny}| = 5$.

A. Entropy of each branch

Wind = Weak (S_{Weak}): Days D1, D8, D9 → [2⁺, 1⁻] → $|S_{Weak}| = 3$

$$Entropy(S_{Weak}) = -\frac{2}{3}\log_2\frac{2}{3} - \frac{1}{3}\log_2\frac{1}{3} = 0.9183$$

Wind = Strong (S_{Strong}): Days D2, D11 → [0⁺, 2⁻]? Wait check dataset:

Sunny, Strong (D2 = No, D11 = Yes) → [1⁺, 1⁻].

So: S_{Strong} = [1⁺, 1⁻] → $|S_{Strong}| = 2$

$$Entropy(S_{Strong}) = -\frac{1}{2}\log_2\frac{1}{2} - \frac{1}{2}\log_2\frac{1}{2} = 1.0$$

B. Calculate $Gain(S_{Sunny}, Wind)$

$$Gain(S_{Sunny}, Wind) = Entropy(S_{Sunny}) - \left[\frac{|S_{Weak}|}{|S|}Entropy(S_{Weak}) + \frac{|S_{Strong}|}{|S|}Entropy(S_{Strong})\right]$$

$$= 0.97 - \left[\frac{3}{5}\cdot 0.9183 + \frac{2}{5}\cdot 1.0\right]$$

$$= 0.97 - (0.551 + 0.4) = 0.97 - 0.951 = \boxed{\mathbf{0.02}}$$

Building Tree using ID3 Algorithm

Humidity split on S_{Sunny}

Subset: S_{Sunny} = Days $\{D1, D2, D8, D9, D11\} \rightarrow |S_{Sunny}| = 5$ with counts $[2^+,\ 3^-]$.

Entropy of the Sunny subset (for reference)

$$Entropy(S_{Sunny}) = -\tfrac{2}{5}\log_2\left(\tfrac{2}{5}\right) - \tfrac{3}{5}\log_2\left(\tfrac{3}{5}\right) = 0.9709505945 \approx 0.9710$$

A. Entropies of Humidity branches

Humidity = High (S_{High}): Days D1, D2, D8 $\rightarrow [0^+, 3^-]$, $|S_{High}| = 3$

$$Entropy(S_{High}) = -\tfrac{0}{3}\log_2\left(\tfrac{0}{3}\right) - \tfrac{3}{3}\log_2\left(\tfrac{3}{3}\right) = 0 - 1 \cdot 0 = \mathbf{0.0}$$

Humidity = Normal (S_{Normal}): Days D9, D11 $\rightarrow [2^+, 0^-]$, $|S_{Normal}| = 2$

$$Entropy(S_{Normal}) = -\tfrac{2}{2}\log_2\left(\tfrac{2}{2}\right) - \tfrac{0}{2}\log_2\left(\tfrac{0}{2}\right) = -1 \cdot 0 - 0 = \mathbf{0.0}$$

B. Information Gain for Humidity

$$Gain(S_{Sunny}, \text{Humidity}) = Entropy(S_{Sunny}) - \left[\frac{|S_{High}|}{|S_{Sunny}|}\,Entropy(S_{High}) + \frac{|S_{Normal}|}{|S_{Sunny}|}\,Entropy(S_{Normal})\right]$$

Substitute values:

$$= 0.9709505945 - \left[\frac{3}{5}\cdot 0.0 + \frac{2}{5}\cdot 0.0\right] = 0.9709505945 - 0$$

$$\boxed{Gain(S_{Sunny}, \text{Humidity}) = 0.97095 \ \ (\approx 0.9710)}$$

Building Tree using ID3 Algorithm

Attribute	**Information Gain**
Humidity	**0.97**
Temperature	0.57
Wind	0.02

$$\boxed{\text{Best attribute for Sunny node} = \text{Humidity}}$$

Building Tree using ID3 Algorithm

For Rain Attribute

A. Entropy of the Rain subset

$$Entropy(S_{Rain}) = -\frac{3}{5}\log_2\frac{3}{5} - \frac{2}{5}\log_2\frac{2}{5} = 0.9710$$

We'll evaluate the remaining attributes:

- Temperature
- Humidity
- Wind

Building Tree using ID3 Algorithm

Sub-branches:

Temp	Days	Class	Count	Entropy
Mild	D4, D10, D14	[2^+,1^-]	3	0.9183
Cool	D5, D6	[1^+,1^-]	2	1.0000

$$Gain(S_{Rain}, Temperature) = 0.9710 - \left[\frac{3}{5} \cdot 0.9183 + \frac{2}{5} \cdot 1.0\right]$$

$$= 0.9710 - (0.5510 + 0.4000) = 0.9710 - 0.9510 = \boxed{\mathbf{0.02}}$$

Attribute: Temp on SRain

Sub-branches:

Humidity	Days	Class	Count	Entropy
High	D4, D5, D10	[2^+,1^-]	3	0.9183
Normal	D6, D14	[1^+,1^-]	2	1.0000

$$Gain(S_{Rain}, Humidity) = 0.9710 - \left[\frac{3}{5} \cdot 0.9183 + \frac{2}{5} \cdot 1.0\right]$$

$$= 0.9710 - 0.9510 = \boxed{\mathbf{0.02}}$$

Attribute: Humidity on SRain

Building Tree using ID3 Algorithm

Attribute: Wind on SRain

Sub-branches:

Wind	Days	Class	Count	Entropy
Weak	D4, D5, D10	$[3^+,0^-]$	3	0.0
Strong	D6, D14	$[0^+,2^-]$	2	0.0

$$Gain(S_{Rain}, Wind) = 0.9710 - \left[\frac{3}{5} \cdot 0 + \frac{2}{5} \cdot 0\right]$$

$$= 0.9710 - 0 = \boxed{\mathbf{0.9710}}$$

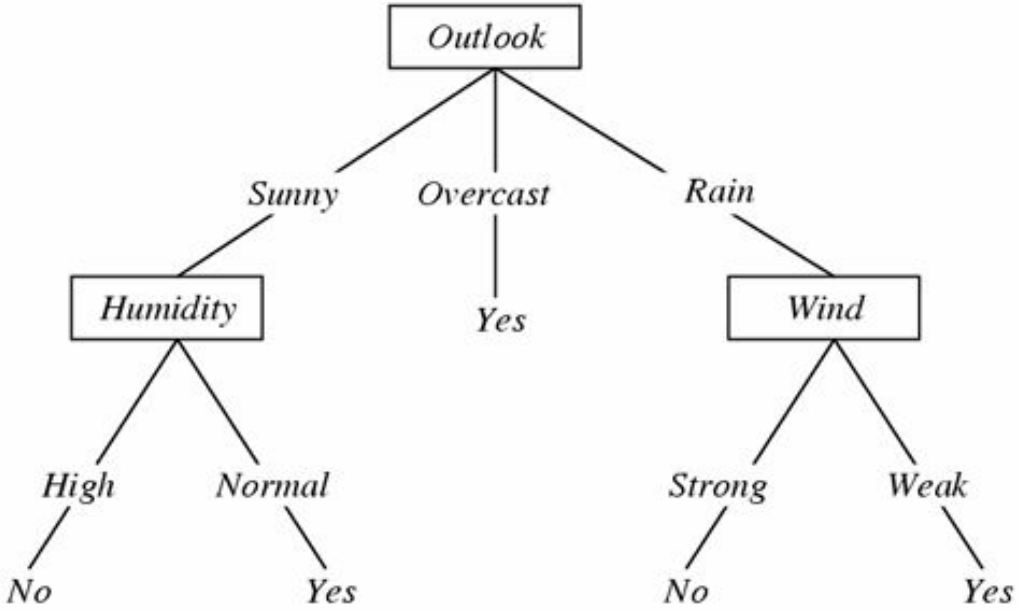
Building Tree using ID3 Algorithm

Attribute	**Information Gain**
Temperature	0.02
Humidity	0.02
Wind	**0.9710**

$$\boxed{\text{Best attribute for Rain node} = \text{Wind}}$$

Rain
- ├─ Wind = Weak → Play = Yes
- └─ Wind = Strong → Play = No

The final Tree structure



EXAMPLE 2

age	income	student	Credit rating	Buys computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31…40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31…40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31…40	medium	no	excellent	yes
31…40	high	yes	fair	yes
>40	medium	no	excellent	no

Methods of Expressing Attribute Test Conditions

An **attribute test condition** is the **rule or question** used at each internal node of a decision tree to **split the data** into smaller subsets.

An **attribute test condition** is the *question asked at each node* of a decision tree that decides **which way a record goes** next.

Purpose

- They **guide how the tree grows** and what branches form.
- Help in **classifying** data correctly by grouping similar examples.
- The **choice of attribute and type of test** affects:
    - Accuracy of the tree
    - Size (depth) of the tree
    - Interpretability of the results

Example in Context

If the node tests **Temperature ≤ 75**,

- Records with Temperature ≤ 75 go to the **left child**.
- Records with Temperature > 75 go to the **right child**.

Methods of Expressing Attribute Test Conditions

The **form of the test condition** depends on the **type of attribute**:

- Binary
- Nominal (Categorical)
- Ordinal
- Continuous (Numeric)

Attribute Type	Test Condition Form	Example Split
Binary	Binary	Windy?
Nominal	Multiway / Binary	Outlook = Sunny
Ordinal	Ordered Binary	Temp ≤ Mild
Continuous	Numeric Threshold	Humidity ≤ 80

Methods of Expressing Attribute Test Conditions

1. Binary Attributes

•Have only two values, like Yes / No or True / False.
•Test example: Windy? → Yes / No

2. Nominal Attributes

•Have multiple categories with no order, like Outlook = Sunny, Rainy, Overcast.
•Can split:

- One branch for each value (multiway)
- Or group some values together (binary)

3. Ordinal Attributes

•Have a natural order, like Temp(Cold) < Temp(Mild) < Temp(Hot).
•Test uses order comparison, example: Temperature ≤ Mild → Yes / No

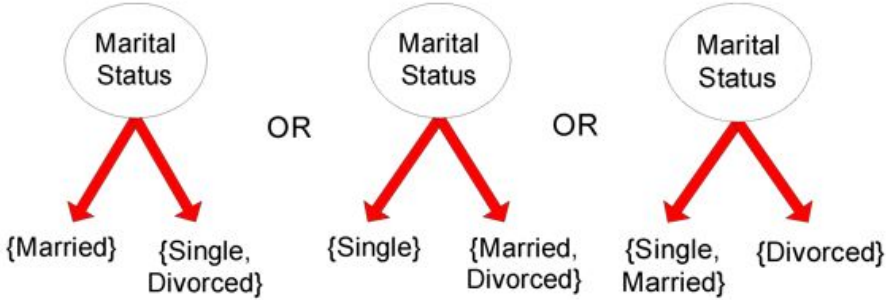
4. Continuous Attributes

•Numeric values like Age, Salary, or Humidity.
•Split based on a threshold, example: Temperature ≤ 75 → Yes / No

Test Conditions for Nominal Attributes

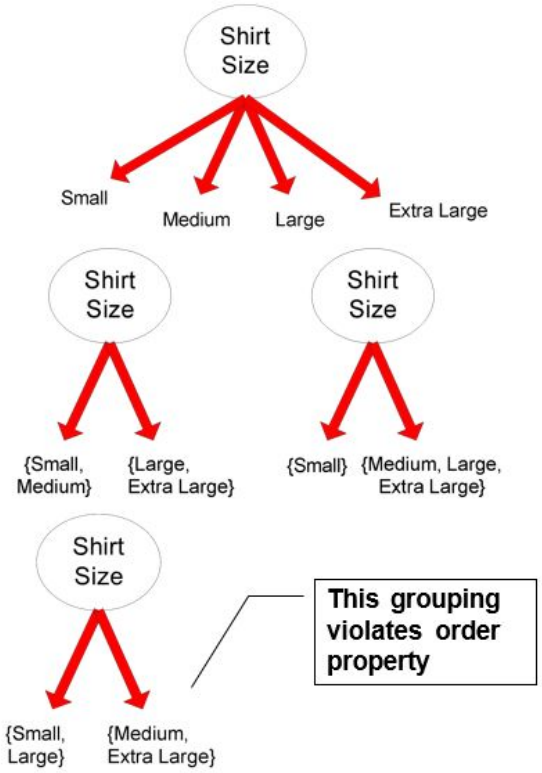
- Multi-way split:
  - Use as many partitions as distinct values.

- Binary split:
  - Divides values into two subsets



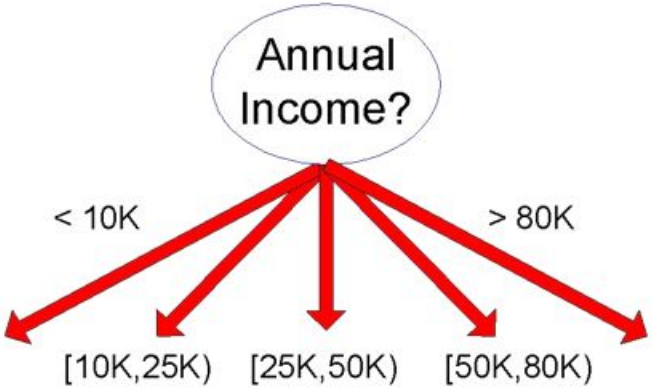
Test Conditions for Ordinal Attributes

- Multi-way split:
  - Use as many partitions as distinct values
- Binary split:
  - Divides values into two subsets
  - Preserve order property among attribute values



Test Conditions for Continous Attributes

(i) Binary split



(ii) Multi-way split

Measures for Selecting an Attribute Test Condition

- The main goal in decision tree learning is to **select the attribute** that best **classifies the training examples**.
- At each node, the algorithm must decide:
  "Which attribute should I test next?"
- The choice is guided by a **selection measure** that evaluates the **quality of the split.**

A good attribute test condition should:
Greatly **reduce uncertainty** about the target class.
Create **child nodes** that are as **pure** (homogeneous) as possible.
Lead to a **smaller, more accurate** decision tree.

Key Measures Mentioned in Tom Mitchell
Tom Mitchell's textbook discusses the following:

1. **Information Gain** — used by ID3.
2. **Gain Ratio** — used by C4.5.
3. **Gini Index** — used by CART.

All three measure **how well an attribute separates** the training examples into target classes.

$$\begin{aligned}
\text{Entropy}(t) &= -\sum_{i=0}^{c-1} p(i|t)\log_2 p(i|t), \\
\text{Gini}(t) &= 1-\sum_{i=0}^{c-1}[p(i|t)]^2, \\
\text{Classification error}(t) &= 1-\max_i[p(i|t)],
\end{aligned}$$

where c is the number of classes and $0\log_2 0 = 0$ in entropy calculations.

Measures for Selecting an Attribute Test Condition

Misclassification Error

- **Used for:** Measuring **impurity or error** in a node (how "mixed up" the classes are).
- **Formula:**

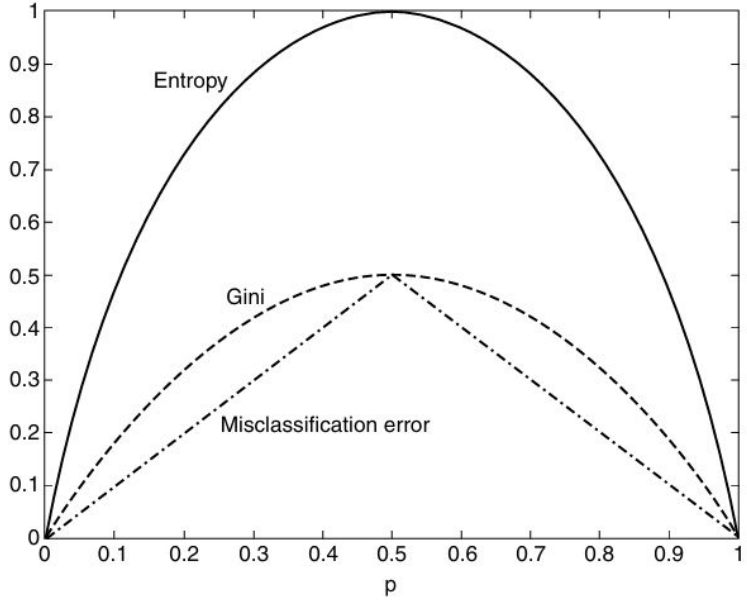
$$\text{Misclassification Error} = 1 - \max(p_i)$$

where p_i = proportion of the majority class in the node.

Meaning:

- If all samples belong to one class → error = 0 (pure node)
- If classes are evenly mixed → error is higher.

Measures for Selecting an Attribute Test Condition



Comparison among the impurity measures for binary classification problems.

Figure: compares the values of the impurity measures for binary classification problems. p refers to the fraction of records that belong to one of the two classes. Observe that all three measures attain their maximum value when the class distribution is uniform (i.e., when p =0.5). The minimum values for the measures are attained when all the records belong to the same class (i.e., when p equals 0 or 1).

Node N_1	Count
Class=0	0
Class=1	6

$$\text{Gini} = 1 - (0/6)^2 - (6/6)^2 = 0$$
$$\text{Entropy} = -(0/6)\log_2(0/6) - (6/6)\log_2(6/6) = 0$$
$$\text{Error} = 1 - \max[0/6, 6/6] = 0$$

Node N_2	Count
Class=0	1
Class=1	5

$$\text{Gini} = 1 - (1/6)^2 - (5/6)^2 = 0.278$$
$$\text{Entropy} = -(1/6)\log_2(1/6) - (5/6)\log_2(5/6) = 0.650$$
$$\text{Error} = 1 - \max[1/6, 5/6] = 0.167$$

Node N_3	Count
Class=0	3
Class=1	3

$$\text{Gini} = 1 - (3/6)^2 - (3/6)^2 = 0.5$$
$$\text{Entropy} = -(3/6)\log_2(3/6) - (3/6)\log_2(3/6) = 1$$
$$\text{Error} = 1 - \max[3/6, 3/6] = 0.5$$

Gini Index

The Gini index measures impurity in a dataset, while the weighted average formula calculates the overall impurity of a split by averaging the Gini indices of its child nodes.

The Gini Index is calculated using

Where pi is the proportion of class I and k is the total number of class.

$$Gini = 1 - \sum_{i=1}^{k} p_i^2$$

The weighted average of the Gini index for a split is calculated by summing the Gini index of each child node multiplied by the proportion of samples that went to that child node.

$$Gini_{split} = \sum_{j=1}^{m} \left(\frac{N_j}{N}\right) \times Gini_j$$

Where:

- Nj → Number of samples in child node *j*
- N → Total number of samples in the parent node
- Ginij → Gini index of child node *j*

Gini Index

How they are used in decision trees

1. **Calculate Gini for child nodes:** For each potential split, the Gini index is calculated for the resulting child nodes.
2. **Calculate weighted average:** The Gini indices of the child nodes are then combined using the weighted average formula to get a single Gini score for that split.
3. **Choose the best split:** The split that results in the **lowest Gini index** (meaning the most "pure" child nodes) is chosen as the next branch in the decision tree.

Gini Index

Weekend	Weather	Parents	Money	Decision
W1	Sunny	Yes	Rich	Cinema
W2	Sunny	No	Rich	Tennis
W3	Windy	Yes	Rich	Cinema
W4	Rainy	Yes	Poor	Cinema
W5	Rainy	No	Rich	Stay In
W6	Rainy	Yes	Poor	Cinema
W7	Windy	No	Poor	Cinema
W8	Windy	No	Rich	Shopping
W9	Windy	Yes	Rich	Cinema
W10	Sunny	No	Rich	Tennis

Gini Index

Step 1: Find Overall Gini Index (Parent Node)

There are **4 possible decisions**:

- Cinema = 6
- Tennis = 2
- Stay In = 1
- Shopping = 1

Total = 10

$$Gini(S) = 1 - \sum p_i^2 = 1 - \left[\left(\frac{6}{10}\right)^2 + \left(\frac{2}{10}\right)^2 + \left(\frac{1}{10}\right)^2 + \left(\frac{1}{10}\right)^2\right]$$

$$Gini(S) = 1 - (0.36 + 0.04 + 0.01 + 0.01) = 1 - 0.42 = 0.58$$

Gini Index

Step 2: Example – Split by "Weather"

Weather	Count	Cinema	Tennis	Stay In	Shopping
Sunny	3	1	2	0	0
Rainy	3	2	0	1	0
Windy	4	3	0	0	1

Sunny:

$$Gini = 1 - [(1/3)^2 + (2/3)^2] = 1 - (0.11 + 0.44) = 0.44$$

Rainy:

$$Gini = 1 - [(2/3)^2 + (1/3)^2] = 1 - (0.44 + 0.11) = 0.44$$

Windy:

$$Gini = 1 - [(3/4)^2 + (1/4)^2] = 1 - (0.56 + 0.06) = 0.38$$

Gini Index

Step 3: Weighted Gini for Split

$$Gini_{Weather} = \frac{3}{10}(0.44) + \frac{3}{10}(0.44) + \frac{4}{10}(0.38)$$

$$Gini_{Weather} = 0.132 + 0.132 + 0.152 = 0.416$$

Conclusion

- Parent Gini = 0.58
- After splitting by Weather = 0.416

→ Reduction in Gini = 0.164

Note: Same has to be continue for all other attributes, until the decision tree is formed. At each stage, the attribute having low gini index will be selected as the best choice.

Gain Ratio

Gain Ratio is a metric used in **Decision Tree algorithms (like C4.5)** to choose the best attribute for splitting the data — it improves upon **Information Gain** by reducing its bias toward attributes with many distinct values.

$$\text{Gain Ratio} = \frac{\text{Information Gain}}{\text{Split Information}}$$

Where

- **Information Gain (IG)** = Reduction in entropy after splitting the dataset on an attribute.
- **Split Information (SI)** = Measures how broadly and evenly the attribute splits the data.

$$\text{Split Information} = -\sum_{i=1}^{k} \frac{|S_i|}{|S|} \log_2 \left(\frac{|S_i|}{|S|} \right)$$

Here:

- $|S_i|$: Number of samples in subset i after the split
- $|S|$: Total number of samples before the split

Gain Ratio

Interpretation

•A high **Information Gain** alone might favor attributes with many unique values (like ID numbers).
•**Gain Ratio** corrects this by dividing IG by Split Info, thus preferring attributes that give **useful, balanced**

•**Gain Ratio = Normalized Information Gain**
It helps prevent **overfitting** and ensures **fair attribute selection**.

•Highest Gain ratio – Best attribute

Algorithm for Decision Tree Induction


```
TreeGrowth (E, F)
if stopping_cond(E,F) = true then
    leaf = createNode().
    leaf.label = Classify(E).
    return leaf.
else
    root = createNode().
    root.test_cond = find_best_split(E, F).
    let V = {v|v is a possible outcome of root.test_cond }.
    for each v ∈ V do
        E_v = {e | root.test_cond(e) = v and e ∈ E}.
        child = TreeGrowth(E_v, F).
        add child as descendent of root and label the edge (root → child) as v.
    end for
end if
return root.
```

Characteristics of Decision Tree Induction

1. **Non-parametric Model**
   No need to assume data distribution (unlike probabilistic models).
2. **NP-Complete Problem**
   Finding the optimal tree is hard; heuristics like greedy top-down search are used.
3. **Computationally Efficient**
   Fast to build and classify even with large datasets; classification time ≈ O(tree depth).
4. **Easy to Interpret**
   Small trees are human-readable and provide accuracy comparable to other methods.
5. **Limited Generalization**
   Struggles with certain Boolean functions (e.g., parity problem).
6. **Robust to Noise**
   Performs well even with noisy data, especially with overfitting control methods.
7. **Handles Redundant Attributes Well**
   Ignores strongly correlated attributes, but irrelevant ones may enlarge the tree.
8. **Data Fragmentation Issue**
   Deeper nodes may have too few samples; stop splitting below a threshold.
9. **Subtree Replication Problem**
   Same subtrees can appear multiple times, making the model unnecessarily complex.
10. **Axis-Parallel Decision Boundaries**
    Splits use one attribute at a time, forming rectangular (non-diagonal) decision regions.
11. Studies have shown that the choice of impurity measure has little effect on the performance of decision tree induction algorithms. This is because many impurity measures are quite consistent with each other